Smart HAS - Comunidade Conectada

Fase 6 - Banco de Dados na Sociedade 5.0

Modelagem Oracle, PL/SQL e integracao com o backend Java

Integrante: Yuri Raphael Alves de Sousa
RM: 556163
FIAP - Faculdade de Informatica e Administracao Paulista
2026

1. O projeto e o que mudou nesta fase

O Smart HAS (tambem chamado ComunidadeConectada) conecta moradores de uma mesma regiao para compartilhar recursos: ferramentas, itens de saude, livros, alimentos e eletronicos, por emprestimo, troca ou doacao. O sistema tem tres partes: aplicativo React Native, backend Spring Boot e painel administrativo Angular.

Ate a Fase 5 a persistencia era o Firebase: Firestore para os dados e Firebase Authentication para a identidade. Nesta fase o projeto passou inteiro para Oracle. Isso nao foi so troca de banco: as regras de negocio que antes ficavam espalhadas no Java desceram para dentro do banco, em procedures e functions PL/SQL, e o Java virou uma ponte fina entre a requisicao REST e a chamada da procedure.

O motivo dessa escolha e o mesmo do estudo de caso visto na fase: regra que vale para todo mundo deve ficar em um lugar so. Hoje o aplicativo, o painel e qualquer carga feita direto no banco obedecem a mesma validacao de emprestimo, porque a validacao e a procedure.

2. Parte 1 - Aprimoramento da solucao

A Parte 1 pede evolucao da solucao ja existente. As mudancas abaixo foram feitas nesta entrega e todas estao no repositorio.

2.1 Saida completa do Firebase

- Removidos FirebaseConfig, FirebaseAuthAdminClient, DataSeeder e os tres repositorios Firestore (usuario, recurso e mensagem), alem da dependencia firebase-admin no pom.xml.
- A senha passou a ser responsabilidade do proprio backend: hash BCrypt gravado em SH_USUARIO.DS_SENHA_HASH, verificado no login com BCryptPasswordEncoder. O bean PasswordEncoder foi declarado em SecurityConfig, que antes nao tinha nenhum.
- O login deixou de depender de uma chamada HTTP externa a API Identity Toolkit do Google. Antes, autenticar exigia internet e a resposta de um servico de terceiros; hoje e uma consulta ao Oracle.
- O token JWT emitido continua igual, entao o aplicativo React Native e o painel Angular nao precisaram mudar nada no contrato de autenticacao.

2.2 Persistencia relacional no lugar de documentos

- Quatro repositorios JDBC novos: JdbcUserRepository, JdbcResourceRepository, JdbcMessageRepository e JdbcPainelRepository, todos sobre JdbcTemplate com RowMapper proprio.
- A busca de recursos que antes baixava documentos e filtrava em memoria agora monta WHERE dinamico e pagina no banco com OFFSET ... FETCH NEXT. So trafega o que a tela vai mostrar.
- A conversa do chat tambem saiu da memoria: o par remetente/destinatario virou filtro no WHERE, com indice de apoio em SH_MENSAGEM.
- Categoria deixou de ser texto solto dentro do documento e virou chave estrangeira para SH_CATEGORIA, o que impede categoria inventada.
- Exclusao de usuario passou a ser logica (ST_USUARIO = 'I'), porque emprestimos e avaliacoes precisam continuar existindo para os indicadores fecharem.

2.3 Funcionalidades novas nascidas do modelo

- Emprestimo com ciclo completo: solicitacao, prazo, devolucao e avaliacao, com tela nova no aplicativo (Meus Emprestimos) e secao nova no painel.
- Alertas operacionais: a rotina de vigilancia abre alerta para cada emprestimo vencido, com severidade conforme o atraso.
- Indicador de circularidade por morador, calculado por function e exibido no aplicativo e no painel.
- Ordenacao da vitrine por proximidade usando a funcao de distancia dentro do banco (endpoint /api/resources/proximos).

2.4 Comunicacao segura, respondendo a avaliacao anterior

A devolutiva da fase anterior trouxe um unico ponto de melhoria: a comunicacao ainda nao estava segura, porque o projeto rodava em HTTP e deixava HTTPS/TLS e as regras do Firestore para depois. Os dois lados desse comentario foram tratados nesta entrega.

Regras do Firestore.

Deixaram de existir junto com o Firestore. Os dados agora estao no Oracle, o acesso acontece por um usuario de schema cuja senha fica fora do repositorio, e a autorizacao de cada rota e feita pelo Spring Security. Nao ha mais banco em modo de teste com leitura e escrita liberadas.

HTTPS e TLS.

- O backend passou a ter o perfil https: application-https.properties liga server.ssl, sobe na porta 8443 e aceita apenas TLS 1.2 e 1.3.
- O certificado e autoassinado e criado na propria maquina por config/gerar_certificado.bat. Ele nao entra no repositorio; em producao entra um certificado emitido por autoridade certificadora, apontando SSL_KEYSTORE e SSL_KEYSTORE_PASSWORD, sem mudar mais nada.
- Com TLS ativo as respostas levam Strict-Transport-Security de um ano, incluindo subdominios. Em HTTP o cabecalho fica desligado de proposito, porque ali ele nao protege nada e so daria aparencia de seguranca.
- O painel Angular aponta para https://localhost:8443/api no build de producao e o aplicativo aceita EXPO_PUBLIC_API_URL, entao trocar o transporte nao exige mexer em codigo.
- O ambiente de desenvolvimento continua podendo subir em HTTP, o que evita o aviso de certificado autoassinado durante a demonstracao. A escolha e do perfil, nao do codigo.

CORS.

Aproveitando a mesma frente, a configuracao de CORS saiu do curinga. Antes era allowedOriginPatterns com asterisco junto de allowCredentials habilitado, a combinacao mais permissiva que existe: qualquer site aberto no navegador do morador poderia chamar a API em nome dele. Agora a lista e fechada e vem de smarthas.cors.allowed-origins, com as origens do painel e do aplicativo. Chamada sem cabecalho Origin, como a do aplicativo nativo, nao passa por CORS e continua funcionando.

Verificacao feita com a aplicacao no ar:

```
protocolo negociado: TLSv1.3    cifra: TLS_AES_256_GCM_SHA384
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
Origin http://localhost:4200         -> 200, com Access-Control-Allow-Origin
Origin http://site-malicioso.example -> 403
handshake forcado em TLS 1.1         -> recusado
POST /api/auth/login por HTTPS       -> 200, token emitido
```

2.5 Tratamento de erro ponta a ponta

As procedures recusam operacao invalida com RAISE_APPLICATION_ERROR na faixa -20000 a -20999. O backend traduz esse erro em uma unica classe utilitaria (OracleErrors), que extrai apenas a primeira linha da mensagem e descarta a pilha ORA-06512. O GlobalExceptionHandler transforma o codigo em status HTTP: 404 para registro inexistente, 409 para estado invalido e 400 para o restante. O texto que o morador le na tela e exatamente o texto escrito dentro da procedure.

3. Parte 2 - Modelagem e implantacao no Oracle

3.1 Do modelo de documentos para o modelo relacional

O Firestore guardava tres colecoes (users, resources, messages) com campos repetidos dentro de cada documento. O modelo relacional resolveu isso em nove tabelas, todas com prefixo SH_, seguindo o padrao de nomenclatura por prefixo de coluna usado no material da fase (id_, nm_, ds_, dt_, vl_, qt_, fl_, st_, nr_, url_).

Tres grupos de tabelas:

- Cadastro, que veio da Fase 5: SH_CATEGORIA, SH_USUARIO, SH_RECURSO e SH_MENSAGEM.
- Circulacao, criada nesta fase: SH_EMPRESTIMO, SH_AVALIACAO e a tabela de resumo SH_RESUMO_USUARIO_MES.
- Operacao: SH_ALERTA e SH_LOG_SISTEMA, alimentadas pelas rotinas PL/SQL.

3.2 DER

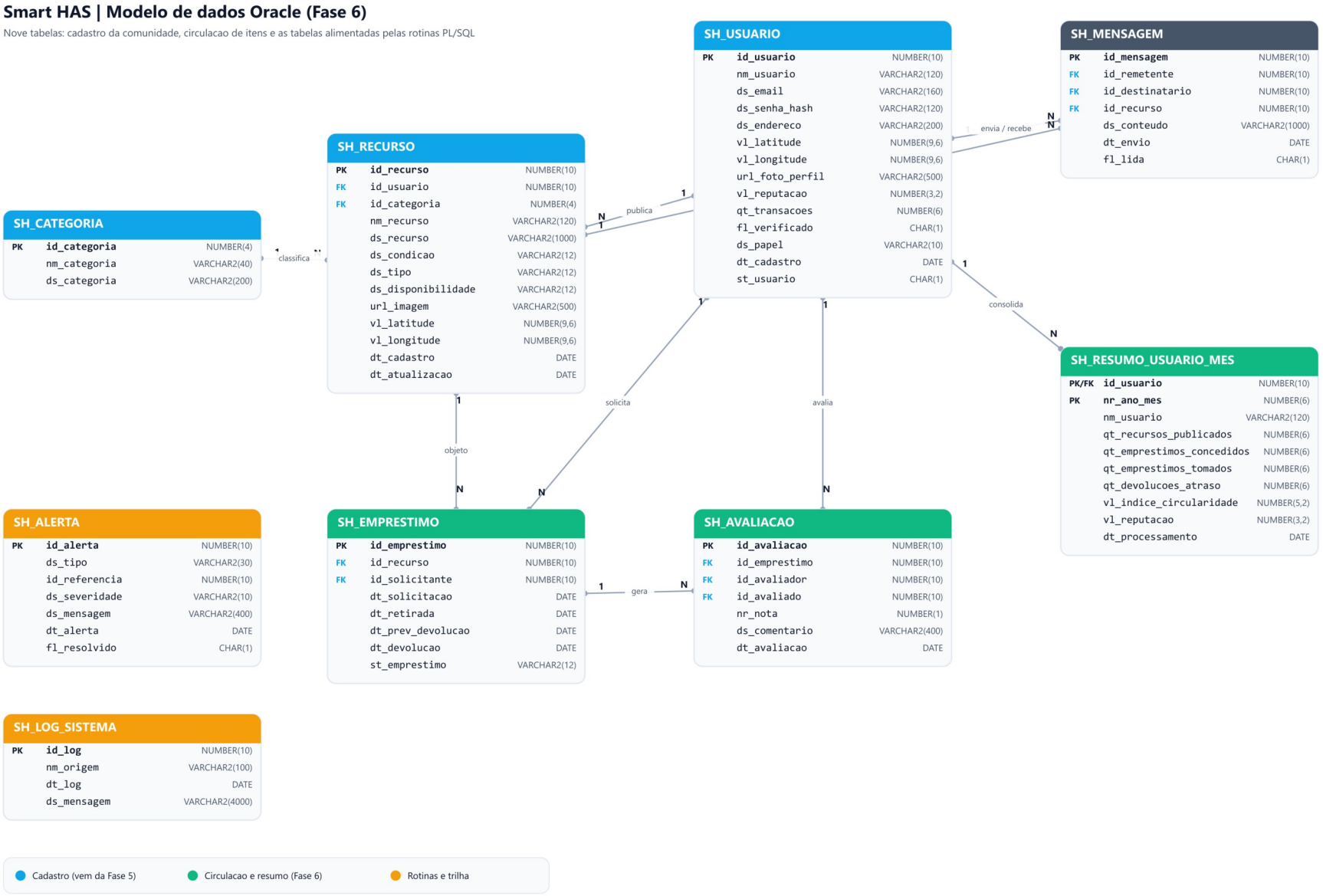


Diagrama entidade-relacionamento do Smart HAS no Oracle. Azul: cadastro. Verde: circulacao e resumo. Laranja: rotinas e trilha.

3.3 Dicionario das tabelas

Tabela	Para que serve	Chave primaria
SH_CATEGORIA	Categorias possiveis de um item, no lugar do texto livre da Fase 5	id_categoria
SH_USUARIO	Morador da comunidade, com hash da senha, coordenadas e reputacao	id_usuario
SH_RECURSO	Item oferecido, com tipo (emprestimo, troca, doacao) e disponibilidade	id_recurso
SH_MENSAGEM	Conversa entre dois moradores, opcionalmente ligada a um item	id_mensagem
SH_EMPRESTIMO	Ciclo do emprestimo: solicitacao, prazo, devolucao e status	id_emprestimo
SH_AVALIACAO	Nota e comentario dados no fim de um emprestimo	id_avaliacao
SH_ALERTA	Alerta aberto pelas rotinas, por exemplo emprestimo vencido	id_alerta
SH_RESUMO_USUARIO_MES	Resumo mensal por morador, recarregado por rotina	id_usuario + nr_ano_mes
SH_LOG_SISTEMA	Trilha de execucao das rotinas, gravada em transacao autonoma	id_log

Regras de integridade que o modelo passou a garantir:

- Chaves estrangeiras ligando recurso a dono e a categoria, emprestimo a recurso e a solicitante, avaliacao a emprestimo e aos dois moradores envolvidos.
- Restricoes CHECK nos dominios: tipo, condicao e disponibilidade do recurso, status do emprestimo, nota de 1 a 5, severidade do alerta e flags S/N.
- Chave unica no email do morador, que no Firestore dependia de uma consulta previa feita pelo proprio codigo.
- Chave primaria composta na tabela de resumo, o que impede duas linhas para o mesmo morador na mesma competencia.

3.4 Scripts e carga

Arquivo	Conteudo
`01_ddl.sql`	9 tabelas, 8 sequences, 6 indices e comentarios de tabela e coluna
`02_seed.sql`	Carga simulada: 6 categorias, 8 moradores, 20 recursos, 15 emprestimos, 10 avaliacoes e 8 mensagens
`03_log.sql`	PRC_SH_GRAVA_LOG, com PRAGMA AUTONOMOUS_TRANSACTION (compilar antes das demais)
`04_functions.sql`	As quatro functions
`05_procedures.sql`	As cinco procedures
`06_consultas.sql`	Consultas de demonstracao, execucao das rotinas e leitura do log
`99_drop.sql`	Remove tudo, ignorando objeto inexistente, para repetir a carga do zero

A carga usa datas relativas a SYSDATE, entao ela continua fazendo sentido em qualquer dia em que for executada. Ha propositalmente dois emprestimos vencidos, para que a rotina de deteccao de atraso tenha o que encontrar na demonstracao, e um emprestimo apenas solicitado, para exercitar a transicao de status.

4. Parte 3 - Functions, procedures e integracao

4.1 Functions

Function	O que devolve	Recursos de PL/SQL
FN_SH_INDICE_CIRCULARIDADE	Indicador: emprestimos concedidos por recurso publicado no mes	%TYPE, validacao de competencia, excecao propria, ZERO_DIVIDE tratado
FN_SH_FICHA_RECURSO	Ficha do item em uma linha ja formatada para a tela	%ROWTYPE, NO_DATA_FOUND e TOO_MANY_ROWS tratados
FN_SH_REPUTACAO	Media das notas recebidas pelo morador	AVG com NVL, NO_DATA_FOUND tratado
FN_SH_DISTANCIA_KM	Distancia em quilometros entre duas coordenadas	Formula de Haversine, DETERMINISTIC, sem DML

A function de indicador e a de dados formatados sao as duas exigidas pelo enunciado. As outras duas existem porque o sistema realmente precisa delas: a reputacao aparece no perfil e a distancia ordena a vitrine.

Nenhuma function faz DML, o que permite chamar todas dentro de um SELECT:

```
SELECT nm_usuario,
       fn_sh_reputacao(id_usuario)                        AS reputacao,
       fn_sh_indice_circularidade(id_usuario, 202609)     AS circularidade
  FROM sh_usuario
 WHERE st_usuario = 'A'
 ORDER BY 3 DESC;
```

4.2 Procedures

Procedure	Papel	Recursos de PL/SQL
PRC_SH_REGISTRA_EMPRESTIMO	Chamada pelo backend quando o morador pede um item	Parametros IN e OUT, IF, quatro excecoes proprias, COMMIT e ROLLBACK
PRC_SH_REGISTRA_DEVOLUCAO	Devolucao com avaliacao opcional e alerta de atraso	IF, excecoes proprias, DUP_VAL_ON_INDEX, atualiza reputacao
PRC_SH_DETECTA_ATRASOS	Rotina de vigilancia: abre alerta para cada emprestimo vencido	CURSOR FOR LOOP, IF de severidade, parametro OUT com a contagem
PRC_SH_GERA_RESUMO_MENSAL	Recarrega a tabela de resumo do mes	BULK COLLECT, FORALL, TABLE OF ...%ROWTYPE, DELETE e INSERT em bloco
PRC_SH_ATUALIZA_REPUTACAO	Grava em SH_USUARIO a reputacao calculada pela function	Reuso de function dentro de procedure, NO_DATA_FOUND
PRC_SH_GRAVA_LOG	Registra a trilha de execucao das demais rotinas	PRAGMA AUTONOMOUS_TRANSACTION, sobrevive ao ROLLBACK da transacao principal

A separacao entre as duas primeiras e as demais e proposital. PRC_SH_REGISTRA_EMPRESTIMO e PRC_SH_REGISTRA_DEVOLUCAO atendem o usuario final e por isso devolvem mensagem pronta para a tela. PRC_SH_DETECTA_ATRASOS e PRC_SH_GERA_RESUMO_MENSAL sao rotinas de operacao, pensadas para rodar em lote, e por isso devolvem apenas a quantidade de linhas afetadas.

Erros de negocio e o status HTTP correspondente

Codigo	Situacao	HTTP
-20201	Recurso ou solicitante inexistente	404
-20202	Solicitante inativo	409
-20203	O solicitante e o proprio dono do item	409
-20204	Item cadastrado como troca ou doacao, nao como emprestimo	409
-20205	Item reservado ou indisponivel	409
-20206	Emprestimo inexistente	404
-20207	Emprestimo ja devolvido ou cancelado	409

-20208	Avaliacao ja registrada por este morador	409
-20101	Competencia fora do formato AAAAMM	400

4.3 O caminho REST para o Oracle

O requisito de acionar uma procedure a partir do backend Java foi atendido no fluxo mais usado do aplicativo, o pedido de emprestimo:

```
POST /api/emprestimos          (EmprestimoController)
  -> EmprestimoService          valida formato e traduz erro
  -> JdbcEmprestimoRepository   SimpleJdbcCall
  -> PRC_SH_REGISTRA_EMPRESTIMO dentro do Oracle
```

A chamada declara os parametros na mao, sem depender da leitura de metadados do dicionario, que exige privilegio extra no schema do aluno:

```
this.chamadaRegistraEmprestimo = new SimpleJdbcCall(jdbc)
        .withProcedureName("PRC_SH_REGISTRA_EMPRESTIMO")
        .withoutProcedureColumnMetaDataAccess()
        .declareParameters(
                new SqlParameter("p_id_recurso", Types.NUMERIC),
                new SqlParameter("p_id_solicitante", Types.NUMERIC),
                new SqlParameter("p_qt_dias", Types.NUMERIC),
                new SqlOutParameter("p_id_emprestimo", Types.NUMERIC),
                new SqlOutParameter("p_ds_retorno", Types.VARCHAR));
```

Outras quatro rotas seguem o mesmo caminho: devolucao, deteccao de atrasos, geracao do resumo mensal e as duas leituras que chamam function dentro de SELECT (indice de circularidade e ficha do recurso).

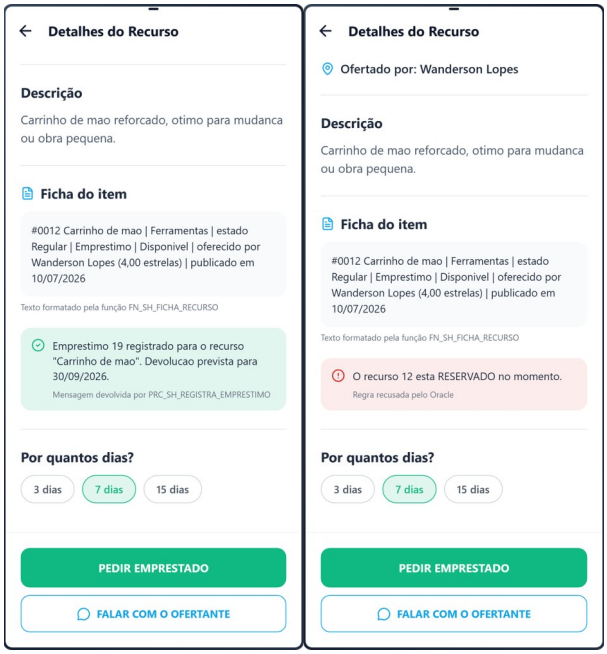
Rota	Objeto no Oracle
POST /api/emprestimos	PRC_SH_REGISTRA_EMPRESTIMO
POST /api/emprestimos/{id}/devolucao	PRC_SH_REGISTRA_DEVOLUCAO
GET /api/emprestimos/indice-circularidade	FN_SH_INDICE_CIRCULARIDADE
GET /api/emprestimos/ficha/{id}	FN_SH_FICHA_RECURSO
GET /api/resources/proximos	FN_SH_DISTANCIA_KM
POST /api/admin/rotinas/detectar-atrasos	PRC_SH_DETECTA_ATRASOS
POST /api/admin/rotinas/resumo-mensal	PRC_SH_GERA_RESUMO_MENSAL
GET /api/admin/alertas	SH_ALERTA (alimentada pela rotina)
GET /api/admin/resumo-mensal	SH_RESUMO_USUARIO_MES (alimentada pela rotina)

4.4 Validacao executada no Oracle da FIAP

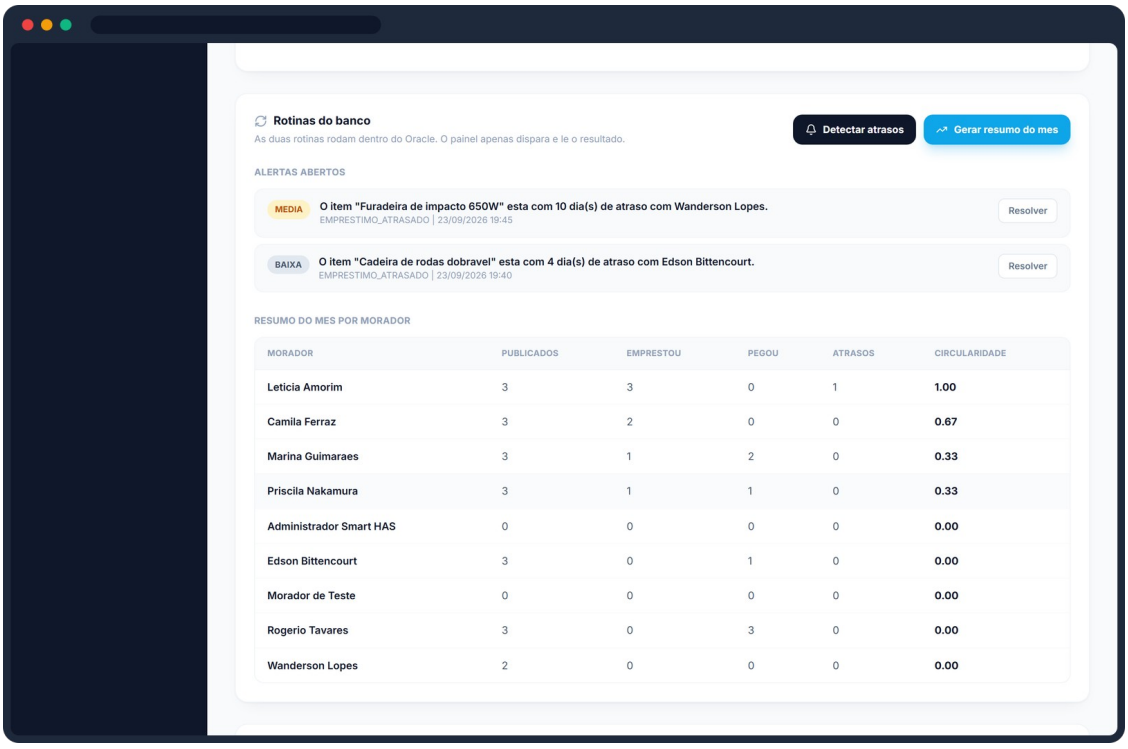
Tudo o que esta descrito acima foi executado no schema RM556163 do servidor oracle.fiap.com.br (Oracle Database 19c Enterprise Edition). Os scripts 01 a 05 rodaram sem erro e o schema terminou com 9 tabelas, 8 sequences, 18 indices, 4 functions e 6 procedures, todos com status VALID.

O que foi verificado	Resultado observado
Carga simulada	6 categorias, 8 moradores, 20 recursos, 15 emprestimos, 10 avaliacoes e 8 mensagens
Functions dentro de SELECT	Reputacao, indice de circularidade, ficha formatada e distancia em km calculados para os moradores da carga
Indicador conferido de duas formas	O valor devolvido pela function e igual ao gravado pela rotina em SH_RESUMO_USUARIO_MES para a mesma competencia
Emprestimo valido	Procedure gravou o registro, reservou o item e devolveu a mensagem com id e data prevista
Regras recusadas	Dono pedindo o proprio item, item reservado, item de doacao, usuario inativo, recurso inexistente e devolucao repetida, cada um com o seu codigo ORA
Rotina de atrasos	Abriu 2 alertas para os emprestimos vencidos da carga, com severidade diferente, e na segunda execucao nao duplicou nenhum
Rotina de resumo	Recarregou a competencia do mes com uma linha por morador
Transacao autonoma	SH_LOG_SISTEMA registrou inicio, fim e erros, inclusive de operacoes que sofreram ROLLBACK

Depois do banco, a verificacao subiu a pilha inteira. O backend conectou no Oracle e respondeu nas rotas de autenticacao (login com hash BCrypt, cadastro novo e e-mail repetido), de emprestimo (solicitacao, recusas com 404, 409 e 400, devolucao e indicador) e do painel (estatisticas, alertas, disparo das duas rotinas e leitura do resumo). O aplicativo React Native e o painel Angular foram usados contra esse mesmo backend, e as telas abaixo saem dessa execucao.



Aplicativo: a mesma tela com o emprestimo aceito e com a regra recusada. Nos dois casos o texto exibido foi escrito dentro da procedure.



Painel: os dois botoes disparam as rotinas PL/SQL, e a tabela abaixo deles e lida pronta de SH_RESUMO_USUARIO_MES.

5. Como executar

5.1 Banco

Conectar no schema Oracle da FIAP (usuario RM + matricula) e executar os scripts na ordem:

```
@01_ddl.sql   @02_seed.sql   @03_log.sql   @04_functions.sql   @05_procedures.sql
```

Para conferir a instalacao e ver as rotinas trabalhando, executar 06_consultas.sql. Para recomecar do zero, 99_drop.sql.

5.2 Backend

As credenciais nao estao no repositorio. Copiar o arquivo de exemplo e preencher:

```
cd backend/smarthas-api
copy config\oracle.properties.example config\oracle.properties
mvnw spring-boot:run
```

O arquivo config/oracle.properties esta no .gitignore. Em vez dele tambem funcionam as variaveis de ambiente ORACLE_URL, ORACLE_USER e ORACLE_PASSWORD. A API sobe em http://localhost:8080 e a documentacao em /swagger-ui.html.

Para subir a mesma API com TLS, gere o certificado local uma vez e use o perfil https:

```
configgerar_certificado.bat
mvnw spring-boot:run -Dspring-boot.run.profiles=https
```

A API passa a responder em https://localhost:8443. O certificado e autoassinado, entao o navegador pede para aceitar a excecao na primeira visita.

5.3 Painel e aplicativo

```
cd web/smarthas-admin   &&   npm install   &&   npx ng serve
cd mobile-react-native/SmartHASApp   &&   npm install   &&   npx expo start --web
```

6. Entregaveis e proximos passos

- Codigo completo no repositorio: https://github.com/YuriRASousa/ComunidadeConectada_FASE5
- Scripts SQL em database/ (DDL, carga, functions, procedures e consultas de demonstracao).
- Apresentacao em PDF, com os dois links na primeira pagina.
- Video de demonstracao (YouTube, nao listado, gravado com a aplicacao ligada no Oracle da FIAP): https://youtu.be/qZQMvaPy4I4

Proximos passos identificados durante a fase:

- Agendar PRC_SH_DETECTA_ATRASOS e PRC_SH_GERA_RESUMO_MENSAL com DBMS_SCHEDULER, para deixarem de depender do clique no painel.
- Agrupar as rotinas em um package, o que daria estado compartilhado e uma compilacao unica.
- Transformar a tabela de resumo em base de um grafico historico no painel, aproveitando que a competencia ja e chave.

7. Conclusao

A fase mudou o centro de gravidade do sistema. Antes, a regra de quem pode pegar um item emprestado existia apenas dentro do Java e valia so para quem passasse por ele. Agora ela mora no Oracle, junto dos dados, e vale para qualquer caminho de entrada. O backend ficou menor e mais previsivel, o modelo relacional passou a impedir estados que o banco de documentos aceitava sem reclamar, e as duas rotinas de operacao deram ao painel numeros prontos em vez de contas refeitas a cada abertura de tela.